

Topic 7: Arrays — Pointer Arithmetic, Multidimensional Arrays & Memory Mapping

1. High-Yield Conceptual Theory

1.1 Array Properties and Memory Layout

An array in C is a contiguous block of homogeneous memory locations:

- **Random Access:** Any element can be accessed in $O(1)$ time using its index.
- **Static Allocation:** Bound at compile time (unless using C99 Variable-Length Arrays or dynamic allocation).
- **Zero-based Indexing:** The first element is at index 0, and the last is at index $N - 1$ for an array of size N .

1.2 Array Decay and Exceptions

In most expressions, the name of an array **decays** into a pointer to its first element:

```
int arr[5]; // 'arr' decays to type: int * (points to &arr[0])
```

The Three Strict Exceptions to Array Decay

An array name does **not** decay into a pointer in exactly three scenarios:

1. **As the operand of sizeof :**

- `sizeof(arr)` returns the total size in bytes of the entire array:

$$\text{Total Bytes} = N \times \text{sizeof}(\text{type})$$

2. **As the operand of the address-of operator (&):**

- `&arr` yields a pointer to the **entire array** with type `int (*)[N]` , rather than `int *` .
- While `arr` and `&arr` have the **same numerical address**, their types and pointer arithmetic behaviors are completely different!

3. **When initializing a string literal array:**

- `char str[] = "GATE";` does not decay "GATE" to a pointer; it copies the characters into the array.

1.3 Pointer Arithmetic & Subscript Commutativity

For a pointer `p` of type `T *` :

$$\text{ess}(p + i) = \text{ess}(p) + i \times \text{sizeof}(T)$$

Array Subscript Equivalence

Array subscripting `arr[i]` is defined strictly in terms of pointer addition and dereferencing:

$$arr[i] \equiv *(arr + i) \equiv *(arr + i \times \text{sizeof}(T))$$

Because addition is commutative:

```
int arr[] = {10, 20, 30};
printf("%d", 1[arr]); // Valid C! Prints 20.
```

Pointer Subtraction

If two pointers `p1` and `p2` point to elements of the same array object (or one past the end):

$$p2 - p1 = \text{ess}(p2) - \text{ess}(p1) / \text{sizeof}(T)$$

The result has the signed integer type `ptrdiff_t` (defined in `<stddef.h>`).

1.4 Memory Mapping Formulations

Let:

- B = Base address of the array (0 or `arr`)
- S = Storage size of each element in bytes (`sizeof(T)`)

1. One-Dimensional (1D) Array

For an array with index range $[0, N-1]$:

$$\text{ess}(i) = B + i \times S$$

For standard 0-indexed arrays ($B = \text{arr}$):

$$\text{ess}(i) = \text{arr} + i \times S$$

2. Two-Dimensional (2D) Array: Row-Major Order (RMO)

In **Row-Major Order** (standard in C/C++), elements are placed into contiguous memory row by row.



For an array N (rows, N columns) with 0-based indices ($0 \leq i < N$):

$$ess(i) = (i \times N) \times$$

For an array with arbitrary lower bounds i_1 and i_2 :

- Number of columns: $N_2 = i_2 - i_1 + 1$

$$ess(i) = ((i - i_1) \times N_2 + i_2) \times$$

3. Two-Dimensional (2D) Array: Column-Major Order (CMO)

In **Column-Major Order** (standard in Fortran, MATLAB), elements are stored column by column.

For an array N with 0-based indices (rows, N columns):

$$ess(i) = (i \times N_1) \times$$

For arbitrary lower bounds ($N_1 = i_1 - i_1 + 1$ = number of rows):

$$ess(i) = ((i - i_1) \times N_1 + i_1) \times$$

4. Three-Dimensional (3D) Array: Row-Major Order

For i_2 (planes/layers, i_2 rows, columns) with 0-based indexing:

$$ess(i) = (i \times (i_2 \times i_3) + i_2) \times$$

General -dimensional formula for i_2 in RMO:

$$ess(i_1 i_2 i) = (i_1 \times \prod_{p=1}^i i_p + i_1) \times$$

1.5 Pointer Levels in 2D Arrays

Given:

```
int a[3][4];
```

Expression	Type	Represents	Arithmetic Step (+ 1)
a	int (*)[4]	Pointer to the first row (array of 4 ints)	$\times \text{sizeof}(in) = 1 \text{ bytes}$
&a	int (*)[3][4]	Pointer to the entire 2D array	$\times \times \text{sizeof}(in) = \text{bytes}$
*a or a[0]	int *	Pointer to the first element in row 0	$1 \times \text{sizeof}(in) = \text{bytes}$
**a or a[0][0]	int	The scalar element at row 0, col 0	Numerical scalar value

Multi-Level Dereferencing Equivalence

$$i \equiv *((*(i))$$

- (a + i) : Points to row i (type: int (*)[4]).
- *(a + i) : Decays to pointer to element 0 of row i (type: int *).
- *(a + i) + j : Points to element of row i (type: int *).
- *((*(a + i) + j) : The actual integer element at (i) .

1.6 Array of Pointers vs. Pointer to an Array

Characteristic	int *p[4];	int (*p)[4];
Meaning	Array of 4 pointers to int	Pointer to an array of 4 int s
Size (64-bit)	$\times = 2 \text{ bytes}$	$1 \times = \text{bytes}$
Parentheses	None	Enclosed (*p)
Memory Allocation	Can point to 4 independent rows of varying lengths (Ragged array)	Must point to a contiguous block of 4-element integer arrays
p + 1 Advances by	$\text{sizeof}(in*) = \text{bytes}$	$\times \text{sizeof}(in) = 1 \text{ bytes}$

2. Common GATE Pitfalls & Edge Cases

1. The `&arr + 1` Pointer Jump

```
int a[5] = {1, 2, 3, 4, 5};
int *ptr = (int *)(&a + 1);
printf("%d", *(ptr - 1)); // Prints 5
```

- `&a` has type `int (*)[5]`.
- `&a + 1` advances by $1 \times \text{sizeof}() = 5 \times = 20$ bytes (past the end of the array).
- Casting to `(int *)` and subtracting 1 moves back by 4 bytes, pointing directly to `a[4]` (value 5).

2. Passing Multidimensional Arrays to Functions

In C, when passing a multidimensional array, **all dimensions except the first must be explicitly specified**:

```
void func(int arr[][4], int rows); // LEGAL
void func(int (*arr)[4], int rows); // LEGAL (identical to above)
void func(int arr[][], int rows); // COMPILE ERROR: missing second dimension
```

- **Reason:** The compiler must know the row length ($N =$) to calculate the memory offset for `arr[i][j] = *(arr + i * 4 + j)`.

3. Out-of-Bounds Memory Traversal in 2D Arrays

```
int a[3] = {{1, 2, 3}, {4, 5, 6}};
printf("%d", a[0][4]); // Prints 5 in almost all flat C compilers
```

- In Row-Major Order, `a[0][4]` translates to `*(a + 0) + 4) = *(base + 4)`.
- Element 4 from base is `a`, which is 5.
- Although strictly an out-of-bounds array access by the ISO C standard, linear RMO mapping makes `*(base + i*N + j)` access the flattened index. GATE problems routinely test this pointer linearization.

3. Previous Year GATE Questions with Step-by-Step Solutions

PYQ 1 (GATE CS 2011)

Question:

Consider the following C program:

```
#include <stdio.h>
int main() {
    int a[3] = {
        { 1, 2}, {3, 4} },
        { 5, 6}, {7, 8} },
        { 9, 10}, {11, 12} }
    };
    int *p = (int *)a;
    printf("%d", *(p + 7));
    return 0;
}
```

What is the output printed by the program?

Step-by-step Solution:

1. `a` is a 3D array of dimensions $3 \times 2 \times 2$.
2. Total elements $= 3 \times 2 \times 2 = 12$ integers.
3. In memory, elements are stored contiguously in Row-Major Order:
 - Index 0: 1 (`a[0][0][0]`)
 - Index 1: 2 (`a[0][0][1]`)
 - Index 2: 3 (`a[0][1][0]`)
 - Index 3: 4 (`a[0][1][1]`)
 - Index 4: 5 (`a[1][0][0]`)
 - Index 5: 6 (`a[1][0][1]`)
 - Index 6: 7 (`a[1][1][0]`)
 - Index 7: 8 (`a`)
4. Pointer `p` points to the start of this contiguous block (base).
5. `*(p + 7)` dereferences the element at offset 7, which is 8.

Output: 8

PYQ 2 (GATE CS 2005)

Question:

An array 1020 is stored in Row-Major Order. The base address of the array is 1000. Each element requires bytes of storage. The array is 1-indexed, meaning the row indices range from 1 to 10 and column indices range from 1 to 20. What is the memory address of the element 15?

Step-by-step Solution:

1. Given parameters:

- Base address = 1000
- Element size = bytes
- Dimensions: = 10 rows, $N = 20$ columns
- Lower bounds: $i_1 = i_2 = 1$
- Target indices: $i = 15$

2. Apply the Row-Major formula with lower bounds:

$$\text{ess}(i) = ((i - 1) \times N + (j - 1)) \times \text{Element size}$$

3. Substitute the values:

$$\text{ess}(15) = 1000 + ((15 - 1) \times 20 + (1 - 1)) \times 1 = 1000 + (14 \times 20) = 1000 + 280 = 1280$$

Output: 1280

PYQ 3 (GATE CS 2016)**Question:**

Consider the following C program:

```
#include <stdio.h>
void f(int *p, int *q) {
    p = q;
    *p = 2;
}

int i = 0, j = 1;
int main() {
    f(&i, &j);
    printf("%d %d\n", i, j);
    return 0;
}
```

What is the output printed by the program?

Step-by-step Solution:

1. In `main()`, `i = 0` and `j = 1`.
2. `f(&i, &j)` passes the address of `i` to pointer `p`, and address of `j` to pointer `q`.
 - `p` points to `i`
 - `q` points to `j`
3. Inside `f`:
 - `p = q`: The local pointer `p` is reassigned to point to whatever `q` points to (`j`). This **does not** change pointer in `main` (C is call-by-value for pointers).
 - `*p = 2`: Dereferencing `p` now writes 2 into the memory location pointed to by `p`, which is `j`. So `j` becomes 2.
4. Variable `i` was never modified; `i` remains 0.
5. `printf` outputs: `0 2`.

Output: 0 2

4. Tricky Practice Questions (Edge Cases & Traps)**Practice Question 1: Pointer to Array Type Arithmetic****Question:**

Consider the following C code:

```
#include <stdio.h>
int main() {
    int arr[3][4] = {
        {10, 20, 30, 40},
        {50, 60, 70, 80},
        {90, 100, 110, 120}
    };
    int (*p)[4] = arr;
    p++;
    printf("%d %d", (*p), *(*p + 2));
    return 0;
}
```

What is printed?

Detailed Explanation:

1. `p` is a pointer to an array of 4 integers: `int (*)[4]`.
2. `p = arr;` points `p` to row 0 (`arr[0]`).
3. `p++;` advances `p` by $1 \times \text{sizeof}(o) = \times \text{sizeof}(in)$.
 - `p` now points to row 1 (`arr`).
4. Expression 1: `(*p)`
 - `*p` dereferences to row 1, decaying into `int *` pointing to `arr[0]`.
 - `(*p)` accesses the element at index 1 of row 1, which is `60`.
5. Expression 2: `*( *p + 2)`
 - `*p` is `&arr[0]`.
 - `*p + 2` is `&arr`.
 - `*( *p + 2)` is the value at `arr`, which is `70`.
6. Output: `60 70`.

Output: 60 70

Practice Question 2: Negative Indexing and Offset Arithmetic

Question:

What will be the output of the following C code?

```
#include <stdio.h>
int main() {
    int a[] = {2, 4, 6, 8, 10, 12, 14};
    int *p = a + 4;
    printf("%d %d\n", p[-2], *(p - 3));
    return 0;
}
```

Detailed Explanation:

1. `a` is `{2, 4, 6, 8, 10, 12, 14}`.
2. `p = a + 4;` points to `a[4]`, which holds value `10`.
3. `p[-2]` is defined as `*(p + (-2)) = *(p - 2)`.
 - Offset $-2 = 2$.
 - `a` = `6`.
4. `*(p - 3)` evaluates to offset $-3 = 1$.
 - `a` = `4`.
5. Output: `6 4`.

Output: 6 4

Practice Question 3: Column-Major Addressing with Negative Lower Bounds

Question:

An array `arr[-2...3, 4...8]` is stored in **Column-Major Order**. The base address of `arr` is 2000. Each element takes 2 bytes. What is the address of `arr`?

Detailed Explanation:

1. Identify the bounds and target indices:
 - Row index range: $i_1 = -2$, $i_2 = 3$, $N_1 = 3 - (-2) + 1 = 6$ rows.
 - Column index range: $j_1 = 4$, $j_2 = 8$, $N_2 = 8 - 4 + 1 = 5$ columns.
 - Target: $i = 1$, $j = 1$.
 - Base address = 2000, Element size = 2 bytes.
2. In **Column-Major Order**, columns preceding column are stored first:
 - Number of full columns skipped = $j - j_1 = 1 - 4 = -3$ columns.
 - Each column contains $N_1 = 6$ elements.
 - Total elements from skipped columns = $2 \times -3 = -6$ elements.
3. Elements skipped in the current column:
 - Number of rows skipped = $i - i_1 = 1 - (-2) = 3$ elements.
4. Total elements skipped before `arr`:

$$\text{offset} = 12 - 6 = 6 \text{ elements}$$

5. Address calculation:

$$\text{ess} = \text{ffset} \times \text{elems} \times = 2000 \times 15 \times 2 = 2000 \times 0 = 200$$

Output: 2030

5. Quick Revision & Edge-Case Checklist

- ☐ `arr` decays to `&arr[0]` (type: `T *`), but `&arr` is a pointer to the entire array (type: `T (*)[N]`).
- ☐ `&arr + 1` jumps ahead by the full size of the array: $(N \times \text{sizeof}())$ bytes.
- ☐ Commutativity of subscripts: `arr[i] == *(arr + i) == i[arr]`.
- ☐ In 2D array N :
 - **Row-Major:** $\text{ess}(i) = (i \times N) \times$
 - **Column-Major:** $\text{ess}(i) = (\times i) \times$
- ☐ To pass multidimensional arrays to functions, the size of all dimensions except the first must be known at compile time: `void f(int a[][C])`.
- ☐ `int *p[N]` is an array of N pointers; `int (*p)[N]` is a single pointer to an array of N integers.
- ☐ Pointer subtraction `p2 - p1` produces element count (`ptrdiff_t`), valid only within the same array bounds.

This concludes **Topic 7: Arrays**. Reply with **"Continue"** to proceed to the next topic: **Linked Lists (Singly Linked List, Operations, Edge Cases & Complexity)**.